

Trees

LEARNING OBJECTIVES

After studying this chapter, the readers will be able to:

- Understand and implement a tree
- Perform different operations in a tree
- Understand binary search trees
- Know applications of binary search trees
- Understand and implement AVL trees
- Perform different operations on AVL trees
- Understand and implement splay trees
- Perform different operations on splay trees

We've learned about linear data structures like arrays, stacks and queues up to this point. In this chapter will concentrate on trees, which are non-linear data structures. The list, dictionary, class and queue modules can all be used to create tree data structures. Simple examples of trees include files and folders in a directory, decision-based algorithms, domain name system, representing a family tree and so on.

7.1 TREES

Data structures can be categorized into linear and non-linear structures. A data structure is said to be non-linear if its elements are not in a sequence. In other words, every element in a non-linear data structure does not have a unique successor and unique predecessor. Neither list nor dictionary is useful when we want a hierarchical structure of data, while stacks and queues do reflect hierarchy to some extent. They are, however, limited to one dimension. To overcome this limitation, we define a new data structure – *tree* – which consists of nodes (which are also called vertices) and edges (which are called arcs) with an inverted tree-like structure as shown in Figure 7.1.

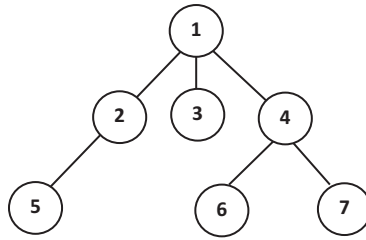


Figure 7.1 Tree

In the non-linear data structure tree, a node can have more than one successor but not more than one predecessor and in a *graph*, a node can have more than one successor and more than one predecessor as shown in Figure 7.2.

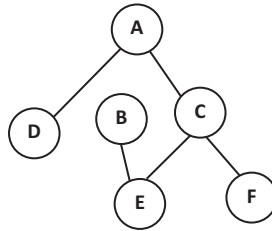


Figure 7.2 Graph

Definition: A tree can be defined as a finite set of one or more nodes such that

- i) One of the nodes in the set, say, R, is identified as the root.
- ii) The rest of the nodes of the set can be partitioned into k number of subsets T₁, T₂, ..., T_k such that each of these subsets also form a tree and are called the sub-trees of the root.

The following terminology is used in the description of a tree:

1. *Root*: the node of the tree that has no parent.
2. *Child*: a node that is directly connected to another node when moving away from the root.
3. *Degree*: the number of edges coming out of the node. Degree of the leaf node is always zero.
4. *Parent*: a node that is directly connected to another node when moving toward the root.
5. *Siblings*: a group of nodes with the same parent.

6. *Descendent*: a node reachable by repeatedly proceeding from parent to child.
7. *Ancestor*: a node reachable by repeatedly proceeding from child to parent.
8. *Leaf node* or *terminal node*: a node with no children.
9. *Internal node* or *non-terminal node*: a node with at least one child.
10. *Forest*: a set of disjoint trees.

In Figure 7.1, Node 1 is the root.

Node 2, Node 3 and Node 4 are children of Node 1; Node 5 is the child of Node 2; and Node 6 and Node 7 are children of Node 3.

Node 1 has degree 3; Node 2 has degree 1; Node 4 has degree 2; and Node 3, Node 5, Node 6 and Node 7 have degree 0.

Node 2 is the parent of Node 5, Node 4 is the parent of Node 6 and Node 7; and Node 1 is parent of Node 2, Node 3 and Node 4.

Node 2, Node 3 and Node 4 are siblings; Node 6 and Node 7 are siblings.

Node 2 and Node 5 are descendants of Node 1.

Node 1 is the ancestor of Node 5 and Node 2.

Node 3, Node 5, Node 6 and Node 7 are leaf nodes.

Node 2, Node 3 and Node 4 are the internal nodes.

7.2 BINARY TREES

A binary tree (BT) is an important type of tree that is often used in computer science and is characterized by the fact that each node can have a left child only, right child only or both (Figure 7.3). In other words, there is no node with more than two branches. (A branch is an edge coming out of the node.)

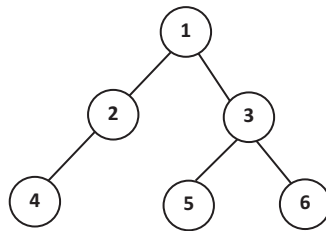


Figure 7.3 Binary tree

7.2.1 Abstract data type of a binary tree

The following are the operations and the set of axioms on the tree data structure:

Operations: See Table 7.1.

Table 7.1 Operations in a tree abstract data type

Operation	Description
create()	Creates an empty binary tree
makeBT(lbt, data, rbt	BT (pointer to the root node [containing data] and lbt and rbt are pointers to the left sub-tree and right sub-tree, respectively, of the root node)
isEmpty()	True if BT = 0, else False
lchild()	BT
rchild()	BT

Axioms: See Table 7.2.

Table 7.2 Axioms in tree abstract data type

Operation	Result
isEmpty(create())	TRUE
isEmpty(makeBT(l, d, r))	FALSE
Lchild(makeBT(l, d, r))	L
Rchild(makeBT(l, d, r))	R
Data(create())	ERROR

7.3 TYPES OF BINARY TREES

Different shapes of binary trees are named as follows:

Left-skewed binary tree: The right sub-tree is missing in every node of a tree (Figure 7.4).

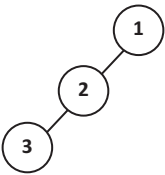


Figure 7.4 Left-skewed binary tree

Right-skewed binary tree: The left sub-tree is missing in every node of a tree (Figure 7.5).

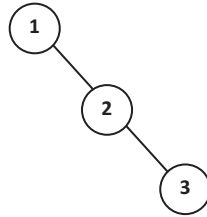


Figure 7.5 Right-skewed binary tree

Strictly binary tree: A binary tree in which every node has zero or two children (Figure 7.6).

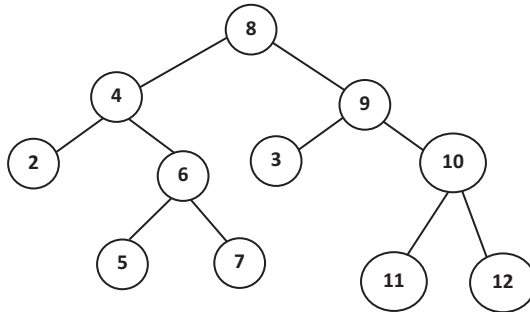


Figure 7.6 Strictly binary tree

Full binary tree: A binary tree in which all the interior nodes have two children and all the leaf nodes are at the same level. In a full binary tree, each node has two children or no child (Figure 7.7).

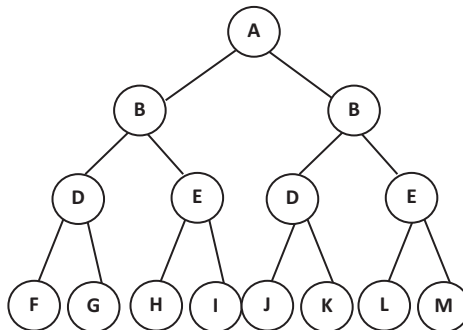


Figure 7.7 Full binary tree

Thus, the number of nodes in a full binary tree is

$$1 + 2 + 2^2 + 2^3 + \dots + 2^h = 2^{h+1} - 1$$

Thus, if n is the total number of nodes in a full binary tree

$$n = 2^{h+1} - 1$$

Therefore, $2^{h+1} = n + 1$ or

$$h = \log_2(n+1) - 1.$$

In Figure 7.7, $h = 3$ and hence there are $2^4 - 1 = 15$ nodes.

Complete binary tree: A binary tree in which all the levels are completely filled except the last level and all the nodes in the last level are as far left as possible (Figure 7.8).

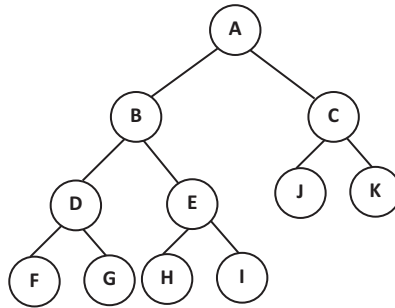


Figure 7.8 Complete binary tree

In other words, in a complete binary tree, the leaf nodes are at the same level or at two consecutive levels. Add children of a node from left to right.

A full binary tree is a complete binary tree but not vice versa.

If the tree is a complete binary tree (not a full binary tree) with n nodes, then

$$\log_2 n < h \leq \log_2(n+1)$$

Theorem: The maximum number of nodes at level i is 2^{i-1} , where $i \geq 1$.

Proof: The root is at level 1. The number of nodes is $2^0 = 1$, which is true.

Assume that the hypothesis is true for level $i - 1$, i.e., the number of nodes at level $i - 1$ is 2^{i-2} . Each node at level $i - 1$ can at maximum have two children.

Therefore, the maximum number of children at level i is $2 \times 2^{i-2} = 2^{i-1}$. Hence the theorem is proved.

Theorem: For any non-empty binary tree, $n_0 = n_2 + 1$, where n_0 is the number of leaves (nodes of degree 0) and n_2 is the number of nodes with degree 2.

Proof: Let n be the total number of nodes in the binary tree. Then, we have

$$n = n_0 + n_1 + n_2 \quad (7.1)$$

Now, every node except the root node has an edge coming into it. Thus, $n - 1 = E$ or $n = E + 1$ where E is the total number of edges.

Total number of edges = n_1 (nodes with degree 1) + n_2 (edges with degree 2) $\times 2$.

$E = n_1 + 2n_2$ which implies

$$n = 1 + n_1 + 2n_2 \quad (7.2)$$

Subtracting Equation 7.2 from Equation 7.1, we get

$$0 = n_0 - 1 - n_2$$

or

$$n_0 = n_2 + 1$$

Consider the tree structure in Figure 7.9.

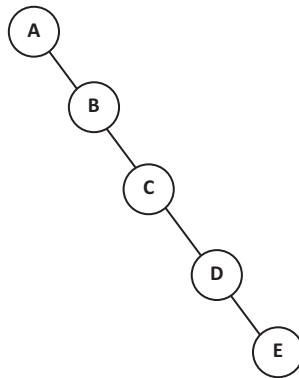


Figure 7.9 Binary tree with single leaf node

Here $n_2 = 0$ and $n_0 = 1$ and the theorem is true (Figure 7.10).

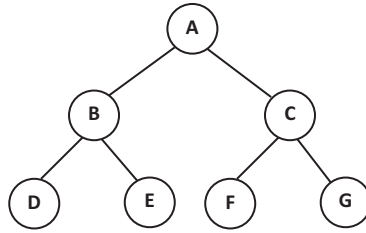


Figure 7.10 Binary tree with double leaf node

Here $n_2 = 3$ and $n_0 = 4$.

Therefore, $n_0 = n_2 + 1$.

Corollary: In a strictly binary tree, the number of leaves (n_0) = number of internal nodes (n_2) + 1.

Theorem: The number of vertices in a binary tree is one more than the number of edges.

Proof: Let there be one vertex in the binary tree. Then there are no edges and the theorem is true.

If there are two vertices, there can only be one edge. Again, the theorem is true. Let the theorem be true for vertices (v) and edges (e). Therefore, $v = e + 1$.

If we add a new vertex to the binary tree, connecting it to a vertex in the binary tree, then we are also adding a new edge. Thus, $v + 1 = e + 2$, which proves that the theorem is true for $v = v + 1$.

7.3.1 Some properties of binary trees

Level: The level of the root node is 1. If a node is at level i , then its children are at level $i + 1$.

Height or depth: The height or depth of a tree is defined as the maximum level of any of the nodes. The depth of a node is the number of edges between the root and the node. The height of a node is the number of edges between the node and the farthest leaf in the sub-tree rooted at the node.

7.4 REPRESENTATION

There are various ways of representing a binary tree. The first method to represent a binary tree can be represented with the help of a list of tuples. As a tuple, the node is declared as a structure with a data field and two

index fields. The index fields contain indices of the tuple in which the left and right children are stored. For example, consider the tree in Figure 7.11.

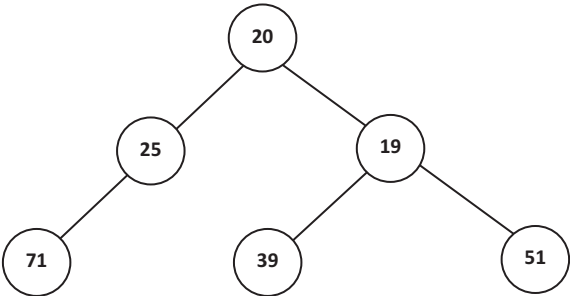


Figure 7.11 Sample binary tree

The root is always stored as the first node. The list of tuples for the tree in Figure 7.11 is shown in a tabular form where each row represents a tuple (Table 7.3).

Table 7.3 Tabular representation of tree

Index	Data	Left index	Right index
0	20	3	4
1	39	None	None
2	51	None	None
3	25	5	None
4	19	1	2
5	71	None	None

Tree = [(20, 3, 4), (39, None, None), (51, None, None), (25, 5, None), (19, 1, 2), (71, None, None)]

The second method to represent a binary tree would be to number the nodes of the tree starting with index 0 as the root node and number all the nodes successively level by level from left to right, considering it as a full binary tree and use these numbers as an index into the list. For example, we represent the tree given in Figure 7.11 as shown in Figure 7.12.

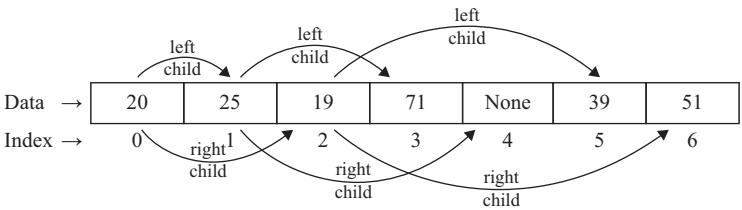


Figure 7.12 Array representation of tree

The *None* cell stores no information. If the tree is heavily skewed to one side there will be many such NULL cells, causing wastage of a lot of memory space. However, the list representation as described earlier has some advantages. For example, given the index of the node in the list, it is easy to find the index of its parent and vice versa. For example, if a node has index i , its children will be at $2i$ and $2i+1$ in the list and its parent will be at $\lfloor i/2 \rfloor$. Since the size of the list can change during the execution of the program, the tree can grow dynamically. A dynamic data structure is more efficient to represent a binary tree, since there is always scope for the tree to expand.

Python program 1

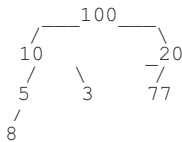
```
# Creating binary tree from given list
from binarytree import build

# List of nodes
nodes = [100, 10, 20, 5, 3, 77, None, 8]

binary_tree = build(nodes)
print('Binary tree from list :\n', binary_tree)
print('\nList from binary tree :', binary_tree.values)
```

Output

Binary tree from list :



List from binary tree : [100, 10, 20, 5, 3, 77, None, 8]

In the dynamic implementation, one more way to represent a node of a tree is an instance of a class composed of the data field and two pointer members, and the binary tree is a collection of these objects organized hierarchically.

7.5 OPERATIONS IN A BINARY TREE

Like any other data structure, insertion, deletion and traversal are the operations we perform in a tree.

7.5.1 Insertion into a binary tree

Insertion is an easy process if the node is to be inserted as a left child or a right child of a leaf node. Here the node containing 'L' is added as a leaf node to the tree in the tree given (Figures 7.13a and 7.13b).

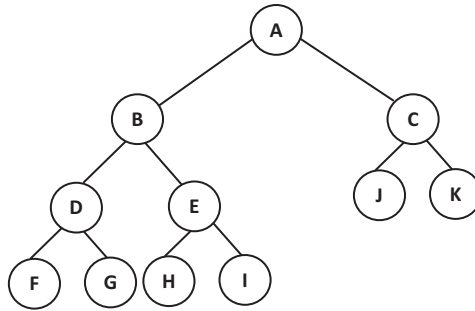


Figure 7.13a Insertion of binary tree

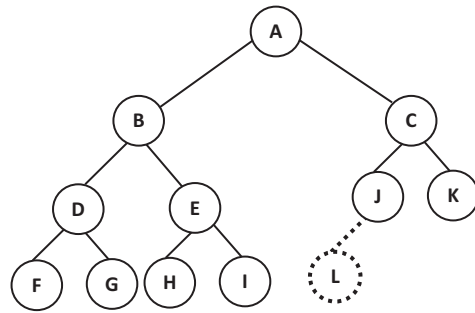


Figure 7.13b Insertion of binary tree

However, if we want to add the new node as an internal node, it becomes ambiguous. We do not know where to add the sub-tree that is replaced by the new node. It may be added as a left child or right of the new node. Therefore, we need a criterion to link this sub-tree. Binary search trees (discussed later in this chapter) provide such a criterion.

7.5.2 Deletion from a binary tree

Deletion is an easy process if the node is to be deleted as a left child or a right child of a leaf node. However, if we want to delete an internal node, it may give rise to ambiguity. When we delete an internal node, the tree will become disconnected. In order to make it connected, a node needs to be replaced for the deleted node. Therefore, we need a criterion to link the sub-tree. Binary search trees (discussed later in this chapter) provide such a criterion.

7.5.3 Binary tree traversals

Information is stored in the nodes of a binary tree. If we want to retrieve the information, we have to traverse the tree, reach that particular node and

extract the data from the node. Traversals can be specified by the ordering of three objects: the current node, its left sub-tree and its right sub-tree. There are several methods to traverse a tree and reach a particular node. For example, we may traverse the left sub-tree of a node first recursively, print the data in the node and then traverse the right sub-tree recursively. Let us call this LDR (Table 7.4).

Table 7.4 Binary tree traversals

DLR	Print Data. Traverse left sub-tree recursively. Traverse right sub-tree recursively.
LRD	Traverse right sub-tree recursively. Traverse left sub-tree recursively. Print Data.
DLR	Print Data. Traverse right sub-tree recursively. Traverse left sub-tree recursively.

Similarly, we can define DLR, DRL, LRD and RLD. Let us apply these traversal methods to the following binary tree (Figure 7.14).

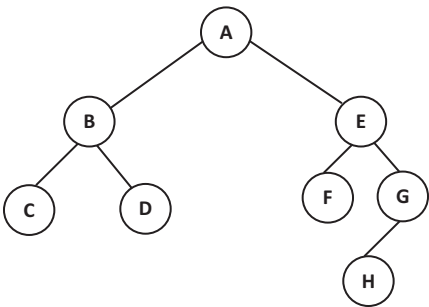


Figure 7.14 Sample binary tree

The following outputs are produced:

LDR:	C B D A F E H G
RDL:	G H E F A D B C
DLR:	A B C D E F G H
RLD:	H G F E D C B A
DRL:	A E G H F B D C
LRD:	C D B F H G E A

We notice that: LDR and RDL are mirror images.

DLR and RLD are mirror images.

DRL and LRD are mirror images.

Ignoring the mirror images (or assuming that the left sub-tree comes before the right sub-tree) we call:

DLR	as pre-order traversal.
LDR	as in-order traversal.
LRD	as post-order traversal.

Pictorially, the traversals can be represented as in Figure 7.15.

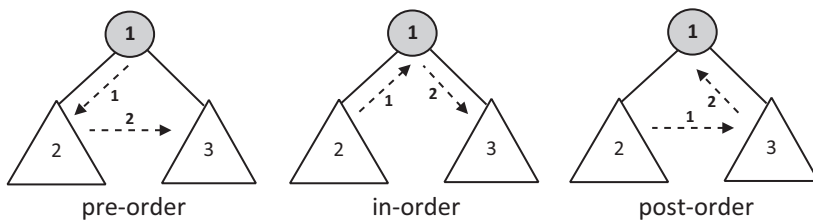


Figure 7.15 In-order, pre-order and post-order traversal

Example: Write the pre-order, in-order and post-order traversal for the given tree in Figure 7.16.

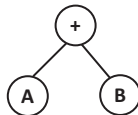


Figure 7.16 Given tree

- i. Pre-order traversal: + A B
- ii. In-order traversal: A + B
- iii. Post-order traversal: A B +

Python program 2

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

Program continues...

```
# Inorder traversal
def inorder(root):
    if root is not None:
        # Traverse left
        inorder(root.left)
        # Traverse root
        print(str(root.key), end=' -> ')
        # Traverse right
        inorder(root.right)
```

Program continues...

```
# preorder traversal
def preorder(root):
    if root is not None:
        # Traverse root
        print(str(root.key) , end=' -> ')
        # Traverse left
        preorder(root.left)
        # Traverse right
        preorder(root.right)
```

Program continues...

```
# postorder traversal
def postorder(root):
    if root is not None:
        # Traverse left
        postorder(root.left)
        # Traverse right
        postorder(root.right)
        # Traverse root
        print(str(root.key) , end=' -> ')
```

Program continues...

```
nodes =[10,20,30,40,50]
print('\nnodes in the binary tree :', nodes)

root = Node(nodes[0])
root.left = Node(nodes[1])
root.right = Node(nodes[2])
root.left.left = Node(nodes[3])
root.left.right = Node(nodes[4])
```

```
print("\nInorder traversal: ", end=' ')
inorder(root)

print("\nPreorder traversal: ", end=' ')
preorder(root)

print("\nPostorder traversal: ", end=' ')
postorder(root)
```

Output

```
nodes in the binary tree : [10, 20, 30, 40, 50]

Inorder traversal: 40 -> 20 -> 50 -> 10 -> 30 ->
Preorder traversal: 10 -> 20 -> 40 -> 50 -> 30 ->
Postorder traversal: 40 -> 50 -> 20 -> 30 -> 10 ->
```

7.5.3.1 Construction of binary tree traversals from known post-order and in-order traversal of a tree

Observe that in a pre-order traversal the root will always be at the first node, and in a post-order traversal the root will always be at the last node.

If two of the three traversals are known, the corresponding binary tree can be constructed. The basic steps for the formation of the tree with known in-order and post-order traversal is as follows:

- Step 1: In the post-order traversal, identify the last node as the root node.
- Step 2: In the in-order traversal, identify the root node somewhere at the middle.
- Step 3: After identifying the root node, the nodes in the left sub-tree and the right sub-tree of the root are the nodes to the left and right of the root node in the in-order traversal.
- Step 4: Repeat steps 1 through 3 to the node of the left sub-tree and the right sub-tree.

Example: Construct the tree of with known in-order and post-order traversals:

In-order traversal:	C B D A F E H G
Post-order traversal:	C D B F H G E A

Solution: From the structure of the post-order traversal, we identify A as the root node. From the in-order traversal, we observe that the nodes C, B, D form the left sub-tree and F, E, G, H form the right sub-tree of the root node (Figure 7.17a).

In-order traversal:	C B D A F E H G
Post-order traversal:	C D B F H G E A

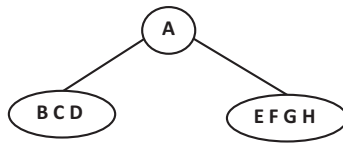


Figure 7.17a Tree with known in-order and post-order traversals

The nodes in the left sub-tree occur in post-order traversal in the order C, D, B indicating that B is the root of the left sub-tree. The nodes F, E, G, H occur in the right sub-tree in the order F, H, G, E, which implies that E is the root of the right sub-tree (Figure 7.17b).

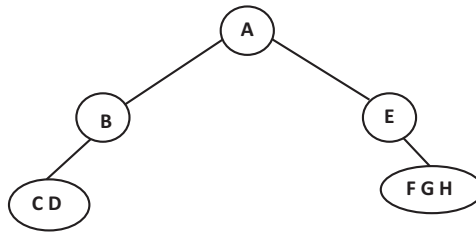


Figure 7.17b Tree with known in-order and post-order traversals

In-order traversal: C B D A F E H G
 Post-order traversal: C D B F H G E A

The nodes B, C, D occur in the order C, B, D showing that C is the left child of B and D is the right child of B. Similarly, we identify F is the left child of E, and G and H constitute the right sub-tree of E (Figure 7.17c).

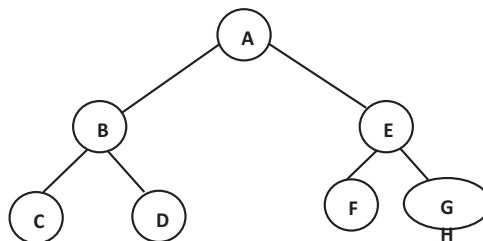


Figure 7.17c Tree with known in-order and post-order traversals

G and H occur in the order H and G in the pre-order traversal. Therefore, G is the root and H is its left child. The full binary tree is as shown in Figure 7.17d.

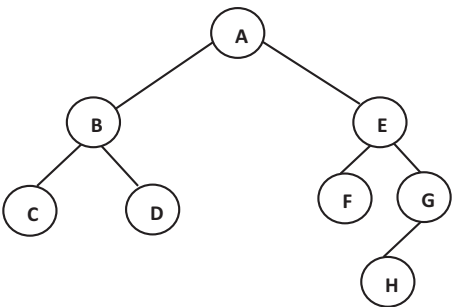


Figure 7.17d Tree with known in-order and post-order traversals

7.5.3.2 Construction of binary tree traversals from known pre-order and in-order traversal of a tree

The basic steps for the formation of the tree with known in-order and pre-order traversals is as follows:

- Step 1: In the pre-order traversal, identify the first node as the root node.
- Step 2: In the in-order traversal, identify the root node somewhere at the middle.
- Step 3: After identifying the root node, the nodes in the left sub-tree and the right sub-tree of the root are the nodes to the left and right of the root node in the in-order traversal.
- Step 4: Repeat steps 1 through 3 to the node of the left sub-tree and the right sub-tree.

Example: Construct a binary tree whose in-order and pre-order traversal are:

In-order traversal	15	20	24	25	33	35	38	40	45
Pre-order traversal	25	20	15	24	35	33	40	38	45

Solution: From the pre-order traversal, we observe that 25 is the root. From the position of 25 in the in-order traversal, we notice that (15 20 24) and (33 35 38 40 45) constitute the left sub-tree and right sub-tree (Figure 7.18a).

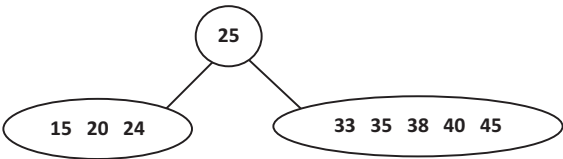


Figure 7.18a Construction of binary tree with given in-order and pre-order traversals

If we observe the three elements 15, 20, 24 in the pre-order traversal, they occur in the order 20, 15, 24, which shows that 20 is the root of the left sub-tree whose left and right children are 15 and 24, respectively. Further, if we observe that the elements constituting the right sub-tree, we notice that 35 is the root and 33 is the left child. Thus, we have so far Figure 7.18b.

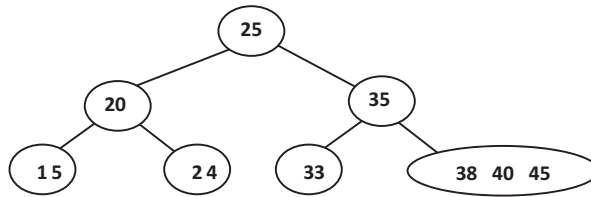


Figure 7.18b Construction of binary tree with given in-order and pre-order traversals

The elements 38, 40, 45 occur in the sequence 40, 38, 45 in the pre-order traversal. Therefore, 40 is the root, with 38 and 45 as the left and right children of 40, respectively. We finally have the tree as shown in Figure 7.18c.

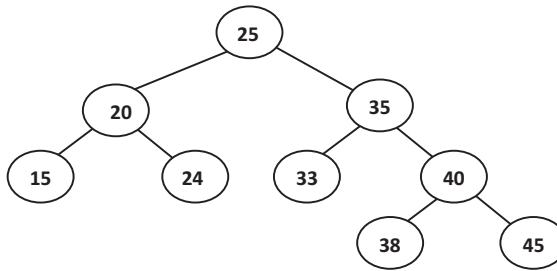


Figure 7.18c Construction of binary tree with given in-order and pre-order traversals

7.6 BINARY SEARCH TREES

The information stored in the nodes of the binary tree will be useful and easily accessible if the data is stored in an organized way. The *binary search tree* (BST) is such a structure where the data stored in the nodes follow a pattern (Figure 7.19).

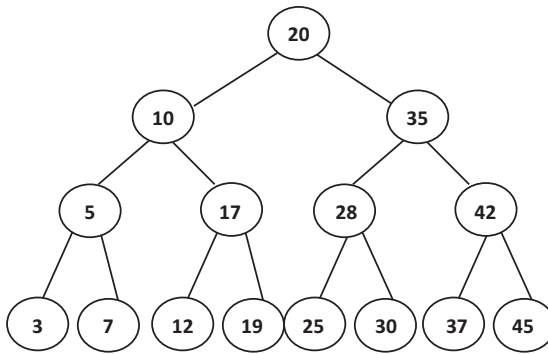


Figure 7.19 Binary search tree

Definition: A binary search tree is a binary tree that is either empty or each node satisfies the following properties:

1. The data value in the left child of the root is less than the data value in the root.
2. The data value in the right child of the root is greater than the data value in the root.
3. The left sub-tree and the right sub-tree of the root are also binary search trees.

Example: Construct the binary search tree as we read the following characters “E D U C A T I O N”.

Step 1: Construct a node with the first character as the root



Figure 7.20a Construction of binary search tree

Step 2: Read the next character D. Construct the node with this character as data. Compare the data with the data in the root. Since it is less (dictionary order) and the left child of the root is NULL, add it as the left child of the root (Figure 7.20b).

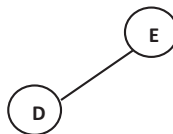


Figure 7.20b Construction of binary search tree

Step 3: The next character read is U. Construct a node with U as data. Since U is greater than E (data in the parent node) and the right child of the root is NULL, add it as the right child of the root (Figure 7.20c).

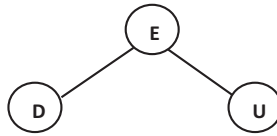


Figure 7.20c Construction of binary search tree

Step 4: We compare the next character C with the value in the root, i.e., E. Since it is less and the left child of the root is not NULL, we make the left child as the root and repeat the procedure. Therefore, we add C as the left child of D (Figure 7.20d).

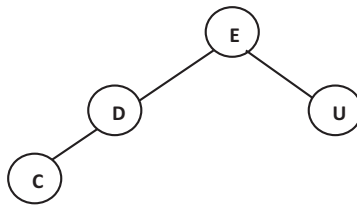


Figure 7.20d Construction of binary search tree

Step 5: We compare the next character A with the value in the root, i.e., E. Since it is less and the left child of the root is not NULL, compare A with D. Since A is less than D and the left child is not NULL, compare it with C. As A is less than C, we add A as the left child of C (Figure 7.20e).

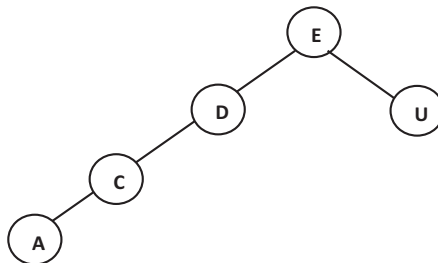


Figure 7.20e Construction of binary search tree

Step 6: We compare the next character T with the value in the root, i.e., E. Since it is greater and the right child of the root is not NULL, compare T with U. Since U is greater than T, we add T as the left child of U (Figure 7.20f).

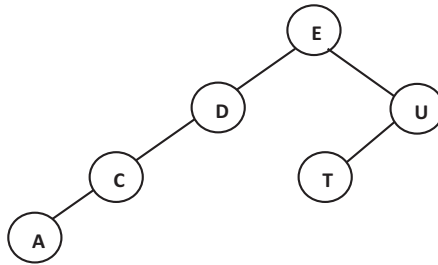


Figure 7.20f Construction of binary search tree

Step 7: We compare the next character I with the value in the root, i.e., E. Since it is greater and the right child of the root is not NULL, compare I with U. Since U is greater than I and the left child is not NULL, compare I with T. Since I is less than T, we add I as the left child of T (Figure 7.20g).

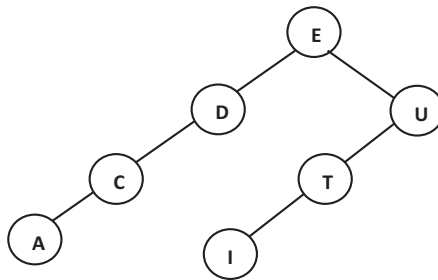


Figure 7.20g Construction of binary search tree

Step 7: We compare the next character O with the value in the root, i.e., E. Since the value is greater and the right child of the root is not NULL, compare O with U. Since U is greater than O and the left child is not NULL, compare O with T. Since O is less than T and the left child is not NULL, compare O with I. Since O' is greater than I, we add O as the right child of I (Figure 7.20h).

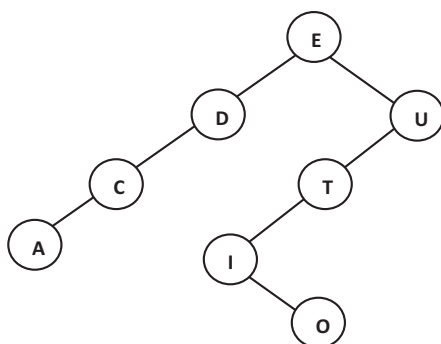


Figure 7.20h Construction of binary search tree

Step 8: We compare the next character N with the value in the root, i.e., E. Since it is greater and the right child of the root is not NULL, compare N with U. Since U is greater than N and the left child is not NULL, compare N with T. Since N is less than T and the left child is not NULL, compare N with I. Since N is greater than I, and the right child is not NULL, compare N with O. Since N is less than O and the left child is NULL, we add N as the left child of O.

Finally, we get the binary search tree as shown in Figure 7.20i.

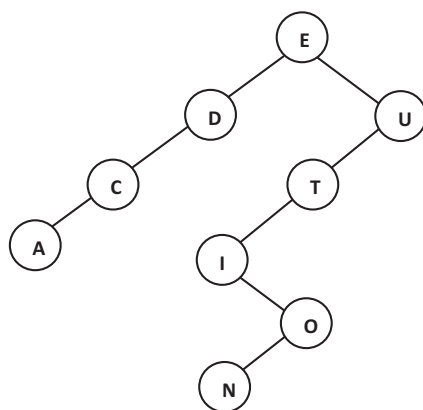


Figure 7.20i Construction of binary search tree

The following program in Python implements the above algorithm using a list to perform operations on a binary search tree.

Python program 3

```
from binarytree import build

# Inserting data into Binary Search Tree
def insert(tree,parent,element,level):
    if element in tree:
        print(element,"\nData already existing in the Binary
        Search Tree\n try any other number")
        return tree

    if (len(tree) < 2**((level+1)-1)):
        for i in range(2**((level+1) - len(tree))):
            tree.append(None)

    if (tree[parent] == None):
        tree[parent] = element
        return tree
    else:
        if (element < tree[parent]):
            parent = parent*2 + 1
            insert(tree,parent, element,level+1)

        else:
            parent=parent*2+2
            insert(tree,parent, element,level+1)
        return tree
```

Program continues

```
def display(tree):
    binary_tree = build(tree)
    print('The Binary Search Tree is :\n', binary_tree)
```

Search

When we want to insert a value in a binary search tree , we have to first check (or search the tree) if that value is already contained in a node since duplicate values are not allowed. We first compare the value with the value contained in the root. If the root contains the value we are searching for, the search is over. If not, we compare the value we are looking for with the value in the root. If it is less, we search the left sub-tree since all the key values in the left sub-tree are less than the key value in the root. If not, we search the right sub-tree. In either case, we search only one sub-tree as in the case of a binary search of a sorted array.

If the element we are searching is less than the “element” in the root, we search the left sub-tree of the root. Else we search the right sub-tree. If found, the pointer to the node is returned. The null pointer is returned if not found.

Example

Suppose we are searching for $x = 46$ in the binary search tree given in Figure 7.21. The path traced by the algorithm is shown by bold arrows, and not found is the result of the search.

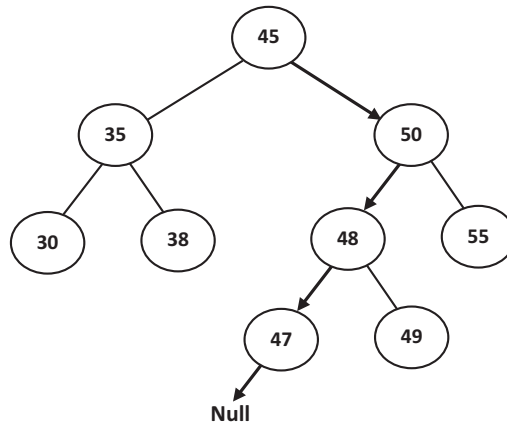


Figure 7.21 Searching in binary search tree

Program continues

```
#Searching a tree
def search(tree,loc, data):
    if (tree[loc] == data) :
        print(data,"Exist in the tree at location ",loc)

    elif(tree[loc] != None and data < tree[loc] ):
        loc = loc*2 + 1
        search(tree,loc, data)
    elif(tree[loc] != None and data > tree[loc] ):
        loc = loc*2 + 2
        search(tree,loc, data)
    else:
        print(data,"Doesnot Exist in the tree")
```

Maximum and minimum

To find the maximum key value in a binary search tree, identify the rightmost node, i.e., the farthest node we can reach following the right branches.

To find the minimum, identify the leftmost node, i.e., the farthest node we can reach by following the leftmost branches.

The following segments of a Python code identify the minimum.

Program continues

```
#Finding the Minimum in a tree
def getRightMin(tree,parent):
    if (tree[parent] != None):
        while(tree[parent*2 +1] != None):
            parent = parent*2+1
        return tree[parent]
    else:
        print("No Data in the tree")
```

Deletion

Three cases arise when we try to delete a node from a binary search tree:

1. The node we want to delete is a leaf node.
2. The node we want to delete has either a left child or a right child but not both.
3. The node we want to delete has both a left child and a right child.

Case 1: The leaf node is deleted, i.e., replaced with NULL. We free the memory that was used by the node we deleted. The node to be deleted is shaded, i.e., the node containing 3 is to be deleted (Figures 7.22a and 7.22b).

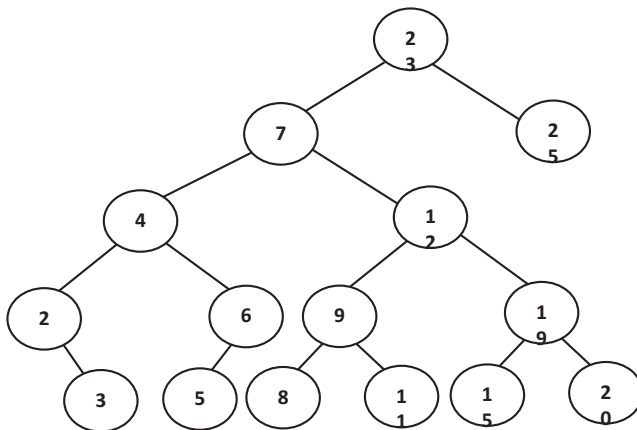


Figure 7.22a Binary search tree before deletion of 3

Case 2: If the node we want to delete has either a left child or a right child, promote the child to the position of the deleted node (Figure 7.22c and 7.22d).

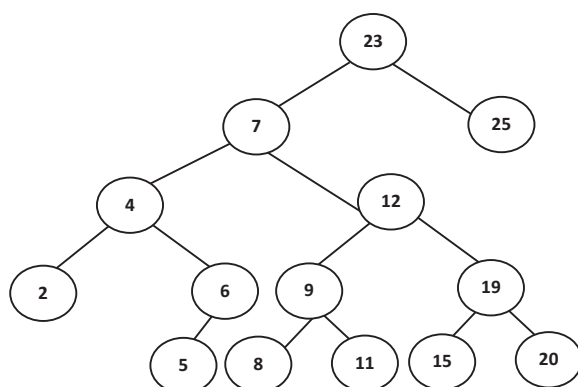


Figure 7.22b Binary search tree after deletion of 3

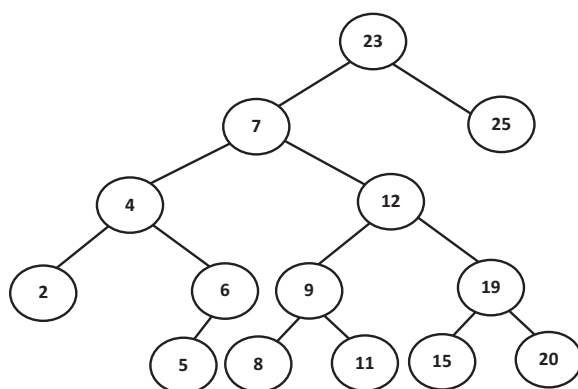


Figure 7.22c Binary search tree before deletion of 6

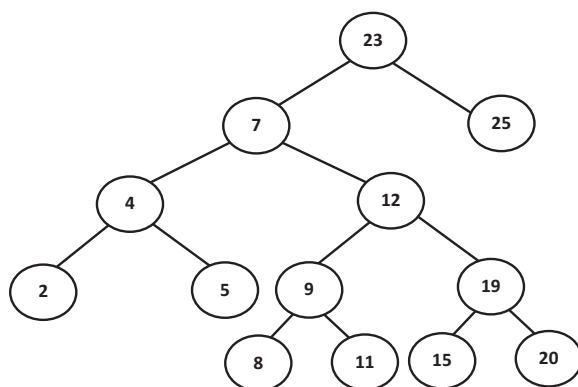


Figure 7.22d Binary search tree after deletion of 6

Case 3: If the node we want to delete has both the children, replace the node with its in-order successor and delete the successor that will be a leaf or a node with only a right child (Figures 7.22e and 7.22f).

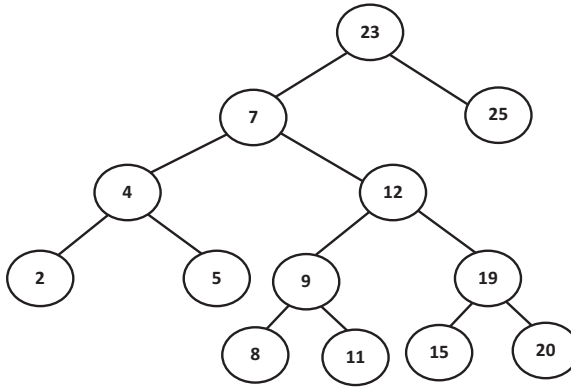


Figure 7.22e Binary search tree before deletion of 7

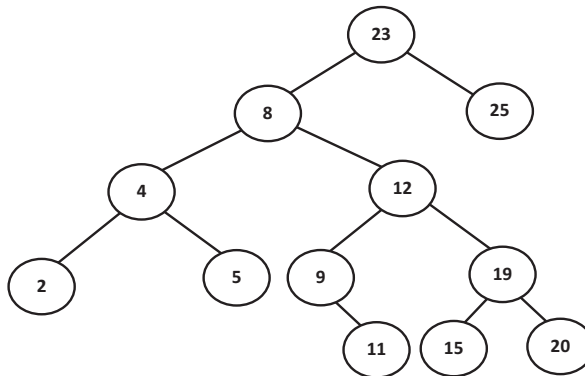


Figure 7.22f Binary search tree after deletion of 7

The following algorithm illustrates the process of deletion for all three cases.

Program continues

```

# Deletion of a Node from the BST
def delNode(tree,parent,data):
    if data in tree:
        i = tree.index(data)
    else:
        print(data,"Does not Exist in the tree")
        return
  
```

```

if(tree[i*2 +1] == None and tree[i*2 + 2] == None):
    tree[i]=None
elif(tree[i*2 +1 ] != None and tree[i*2 + 2] == None):
    tree[i] = tree[i*2 +1 ]
    tree[i*2+1] = None
elif(tree[i*2 +1 ] == None and tree[i*2 + 2] != None):
    tree[i] = tree[i*2 +2 ]
    tree[i*2+2] = None
elif(tree[i*2 +1 ] != None and tree[i*2 + 2] != None):
    min = getRightMin(tree,2*i+2)
    index = tree.index(min)
    tree[i] = min
    tree[index]= None
    if(tree[index*2 +2] != None):
        tree[index]=tree[index*2+2]
        tree[index*2+2]=None

```

Program continues

```

def menu():
    print("\n~~~ M E N U ~~~")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Display")
    print("5. Exit")
    opt = int(input("Enter a valid menu item ... "))
    return opt

```

Program continues

```

if __name__ == "__main__":
    tree = [None]
    level=1
    parent = 0
    choice = 1
    while (choice > 0 and choice < 5 ):
        choice = menu()
        if (choice == 1):
            print("Enter 0 to exit Inserting Data into the BST ")
            x = int(input("Enter an element: "))
            while (x != 0):
                insert(tree,parent,x,level)
                #print(tree)
                x = int(input("Enter an element: "))
            display(tree)

```

```
elif(choice == 2):
    value = int(input("Enter element to search : "))
    search(tree,parent,value)

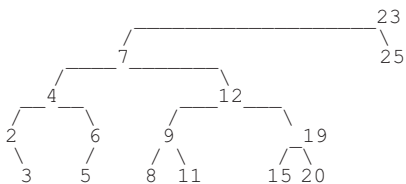
elif(choice == 3):
    value = int(input("Enter element to delete : "))
    delNode(tree,parent,value)
    display(tree)

elif(choice == 4):
    display(tree)

else:
    print("Exit")
```

Output 1

```
~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 1
Enter 0 to exit Inserting Data into the BST
Enter an element: 23
Enter an element: 7
Enter an element: 25
Enter an element: 4
Enter an element: 12
Enter an element: 2
Enter an element: 6
Enter an element: 9
Enter an element: 19
Enter an element: 3
Enter an element: 5
Enter an element: 8
Enter an element: 11
Enter an element: 15
Enter an element: 20
Enter an element: 0
The Binary Search Tree is :
```



Output 2

```

~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 2
Enter element to search : 6
6 Exist in the tree at location 8

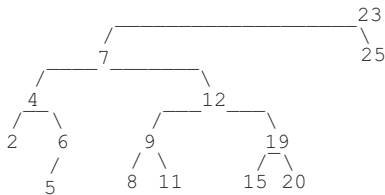
```

Output 3

```

~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 3
Enter element to delete : 3
The Binary Search Tree is :

```

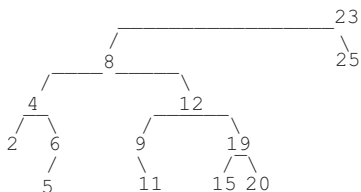


Output 4

```

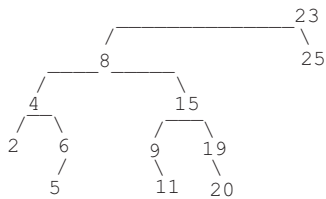
~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 3
Enter element to delete : 7
The Binary Search Tree is :

```



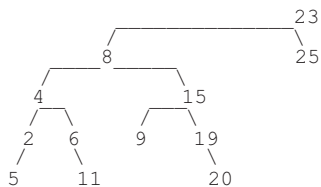
Output 5

```
~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 3
Enter element to delete : 12
The Binary Search Tree is :
```



Output 6

```
~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 3
Enter element to delete : 12
The Binary Search Tree is :
```



Output 7

```
~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 5
Exit
```

The following program in Python implements the above algorithm using a class to perform operations on a binary search tree.

Python program 4

```
# Create a node
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

Program continues

```
#Search
def search(root, val):
    if (root is None or root.key == val):
        return root

    if (root.key < val):
        return search(root.right, val)

    return search(root.left, val)
```

Program continues

```
# Inorder traversal
def inorder(root):
    if root is not None:
        # Traverse left
        inorder(root.left)
        # Traverse root
        print(str(root.key), end=' -> ')
        # Traverse right
        inorder(root.right)
```

Program continues

```
# Insert a node
def insert(node, key):
    temp = search(node, key)
    if (temp != None):
        print("\n", temp.key, " Exists in the BST... Try another Number ")
        return node

    else:
        # Return a new node if the tree is empty
        if node is None:
            return Node(key)
```

```
    if key < node.key:
        node.left = insert(node.left, key)
    else:
        node.right = insert(node.right, key)

    return node
```

Program continues

```
# Find the inorder successor
def minValueNode(node):
    current = node

    while(current.left is not None):
        current = current.left
    return current
```

Program continues

```
# Deleting a node
def deleteNode(root, key):

    if root is None:
        return root

    # Find the node to be deleted
    if key < root.key:
        root.left = deleteNode(root.left, key)
    elif(key > root.key):
        root.right = deleteNode(root.right, key)
    else:
        # If the node is with only one child or
        # no child
        if root.left is None:
            temp = root.right
            root = None
            return temp

        elif root.right is None:
            temp = root.left
            root = None
            return temp

        # If the node has two children,
        # place the inorder successor in position of the
        # node to be deleted
        temp = minValueNode(root.right)

        root.key = temp.key

        # Delete the inorder successor
        root.right = deleteNode(root.right, temp.key)

    return root
```

Program continues

```
def menu():
    print("\n~~~ M E N U ~~~")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Display")
    print("5. Exit")
    opt = int(input("Enter a valid menu item ... "))
    return opt
```

Program continues

```
if __name__ == "__main__":
    root = None
    choice = 1
    while (choice > 0 and choice < 5 ):
        choice = menu()
        if (choice == 1):
            print("Enter 0 to exit Inserting Data into the BST\n ")
            x = int(input("Enter an element: "))
            while (x != 0):
                root = insert(root, x)
                x = int(input("Enter an element: "))
            print("Inorder traversal: ", end=' ')
            inorder(root)

        elif(choice == 2):
            val = int(input("Enter an element to Search: "))
            temp = search(root, val)
            if (temp != None):
                print("\n", temp.key, " Exists in the BST")
            else:
                print("\n", temp.key, "Does not Exist in the BST")

        elif(choice == 3):
            value = int(input("Enter element to delete : "))
            root = deleteNode(root, value)
            print("Inorder traversal: ", end=' ')
            inorder(root)

        elif(choice == 4):
            print("Inorder traversal: ", end=' ')
            inorder(root)

        else:
            print("Exit")
```

Output

```
~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 1
Enter 0 to exit Inserting Data into the BST

Enter an element: 6
Enter an element: 3
Enter an element: 6

6 Exists in the BST... Try another Number
Enter an element: 4
Enter an element: 0
Inorder traversal: 3 -> 4 -> 6 ->

~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 2
Enter an element to Search: 4

4 Exists in the BST

~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 3
Enter element to delete : 3
Inorder traversal: 4 -> 6 ->

~~~ M E N U ~~~
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter a valid menu item ... 4
Inorder traversal: 4 -> 6 ->
```

~~~ M E N U ~~~

1. Insert
2. Search
3. Delete
4. Display
5. Exit

Enter a valid menu item ... 6

Exit

### Successor and predecessor

The successor (predecessor) of a node with key value  $k$  in the binary search tree is the smallest (largest) key that belongs to the tree and is strictly greater than (less than  $k$ ):

- If  $x$  (node containing  $k$ ) has a right child, the successor is the minimum in the right sub-tree of  $x$ .
- Otherwise, the successor is the parent of the farthest node that can be reached by following the right branches backward.

**Example** Consider the binary search tree in Figure 7.23.

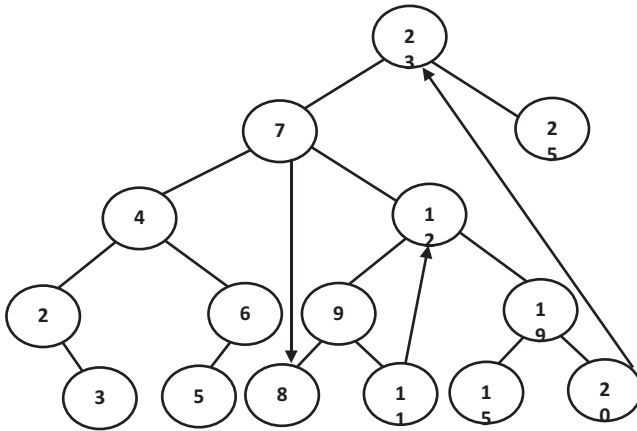


Figure 7.23 Successor and predecessor in a binary search

The successor of 7 is 8 since it is the left-most node of its right sub-tree or the minimum value in its right sub-tree. The successor of 11 is 12. Node 11 has no right sub-tree. Therefore, follow the right branches upward from 11 until there are no more right branches. We reach 9. The parent of 9, which is 12, is the in-order successor of 11. Similarly the successor of 20 is obtained by following the path backward as  $20 \rightarrow 19 \rightarrow 12 \rightarrow 7$  and obtaining the parent of 7, which is 23.

### 7.6.1 Applications of binary search trees

A binary search tree can be used to implement a sorting algorithm. If we insert a set of elements into a binary search tree and make an in-order traversal, we obtain a sorted list (Figure 7.24a).

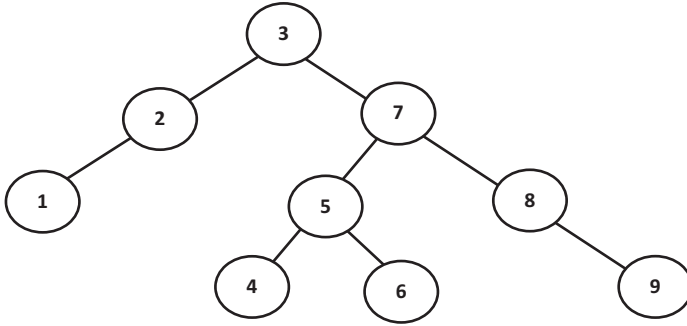


Figure 7.24a Given binary search tree

When we make an in-order traversal, we get the sorted sequence 1 2 3 4 5 6 7 8 9.

If we try to build a binary search tree from an already sorted list, we obtain a skewed binary search tree (Figure 7.24b).

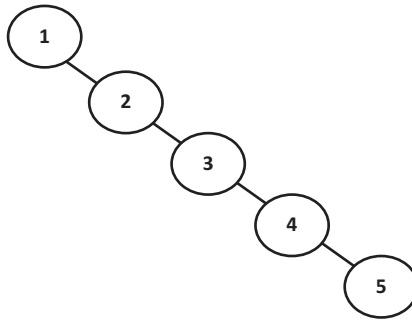


Figure 7.24b Given binary search tree

If there are  $n$  elements in the skewed binary search tree, we need  $n$  steps to reach the last node,  $n-1$  steps to reach the next to last node, etc. Therefore, on the average we need  $n/2$  steps and the search time is  $O(n)$ . We need to keep the height of the tree as small as possible to keep the average search time minimum since the search time is proportional to the height of the tree.

## 7.7 AVL TREES (HEIGHT-BALANCED TREES)

The performance of a search algorithm on a binary search tree depends on the shape of the tree and the shape of the tree depends on the data set. If the data is sorted, the binary search tree would be skewed, and the search would not be efficient as was pointed out in the previous section. On the other hand, if we can build, with the same data set, with minimum height, the search would be faster.

In this section, we define a special type of binary search tree, called the AVL tree (named after the researchers G.M. Adelson-Velsinki and E.M. Landis) in which the tree is nearly balanced. Let us first define a perfectly balanced tree.

**Definition:** A perfectly balanced tree is a binary search tree which is such that:

1. The height of the left sub-tree and right sub-tree of the root is the same.
2. The left sub-tree and the right sub-tree are also perfectly balanced trees.

Figure 7.25 shows a perfectly balanced tree.

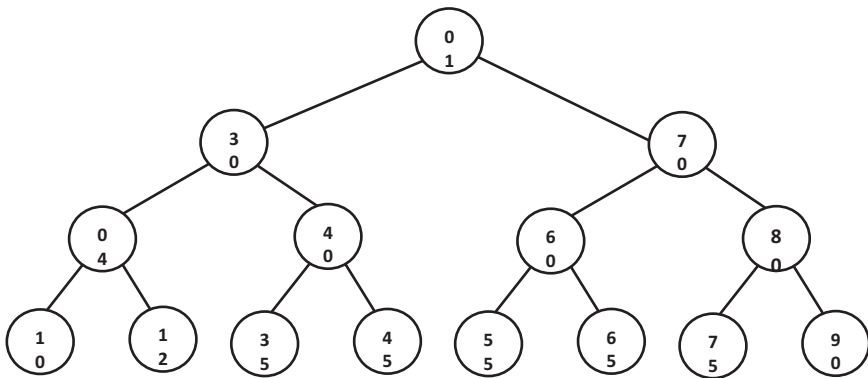


Figure 7.25 Perfectly balanced tree

From the definition of a perfectly binary search tree, we observe that if  $x$  is a node in the tree, the height of its left sub-tree is the same as the height of its right sub-tree. However, we need  $2^h - 1$  nodes to construct such a tree, but this need not be the case always.

**Definition:** An AVL tree (or a height-balanced tree) is a binary search tree such that:

1. The heights of the left sub-tree and right sub-tree of the root differ by at most 1.
2. The left sub-trees and right sub-trees are also AVL trees.

Let  $x$  be a node of the binary search tree. Let  $x_l$  and  $x_r$  be the heights of its left sub-tree and right sub-tree. Then we define the balance factor at node  $x$  as:

$$bf(x) = x_l - x_r$$

The permissible values  $bf(x)$  are  $\{-1, 0, +1\}$  for an AVL tree. Thus, the node in the AVL tree must contain besides the data value and left and right pointers the value of the balance factor.

The *constructor function* initializes the left and right pointers to NULL and the balance factor to 0.

### 7.7.1 Height of an AVL tree

**Proposition:** The height of an AVL tree  $T$  storing  $n$  key values is  $O(\log n)$ .

**Proof:** Let  $n(h)$  be the minimum number of internal nodes in an AVL tree of height  $h$ . We can see that  $n(1) = 0$  and  $n(2) = 1$ . For  $n \geq 3$ , an AVL tree of height contains the root node, an AVL sub-tree of height  $h - 1$  and an AVL sub-tree of height  $h - 2$ . Thus, we can write  $n(h)$  as

$$n(h) = 1 + n(h - 1) + n(h - 2)$$

Obviously,  $n(h - 1)$  is greater than  $n(h - 2)$ . Therefore,

$$\begin{aligned} n(h) &> 2n(h - 2) \\ &\geq 2^2 n(h - 4) \\ &\geq 2^3 n(h - 6) \\ &\geq \dots \\ &\geq \dots \\ &\geq 2^i n(h - 2i) \\ &\geq \dots, \text{ etc.} \end{aligned}$$

Let  $h - 2i = 2$ . Or  $i = (h - 2)/2 = h/2 - 1$ . Therefore,

$$n(h) > 2^{h/2-1}$$

taking logarithms,

$$h/2 - 1 < \log n \text{ or } h < 2\log n + 2.$$

Therefore,  $h$  is  $O(\log n)$ .

Most of the algorithms used for a binary search tree (such as searching, traversals, counting nodes) remain the same. However, insertions and deletions are different since after every insertion and deletion, the AVL property may be lost.

Examples of AVL trees are shown in Figure 7.26.

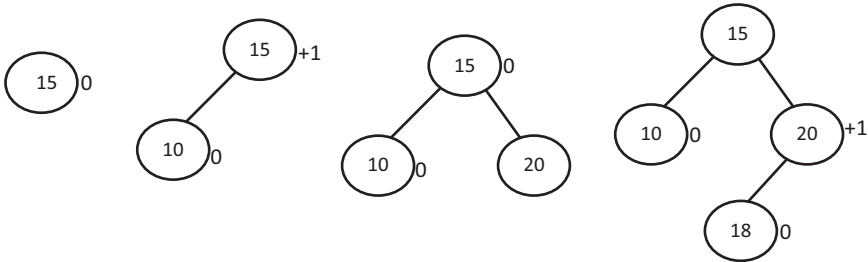


Figure 7.26 Height of AVL tree

### 7.7.2 Insertion

After inserting a node, as is done in any binary search tree, it is necessary to check that each of its ancestors satisfies the AVL property. This is achieved by considering the balance factor at each node. Since the insertion into a binary tree cannot increase the height by more than 1, the balance factor of each node lies in  $\{-2, 2\}$ . If the balance factors of all the nodes lie in  $\{-1, +1\}$ , no further operation is needed. However, if the balance factor at any node is more than  $+1$  or less than  $-1$ , the AVL property is lost and a rebalancing is needed. Let us trace the operations of insertion with an example.

Imagine the situation where we arrive at the structure in Figure 7.27a while constructing a binary search tree.

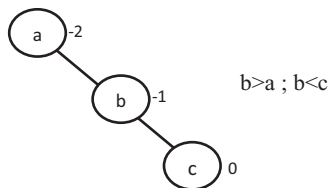


Figure 7.27a Insertion of element in an AVL tree

This does not satisfy the AVL property since the balance factor of the root node is  $-2$ . To fix this we have to perform a rotation. This is done using the following steps.

*LL rotation*

1. b becomes the new root.
2. a takes the ownership of the left child of B.
3. c takes the ownership of the right child of B.

The binary search property is not lost.  
The tree now becomes Figure 7.27b.

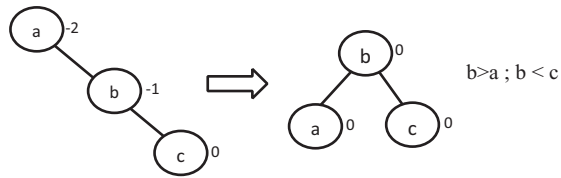


Figure 7.27b Insertion of element in an AVL tree

Suppose we have the mirror image situation of Figure 7.27c.

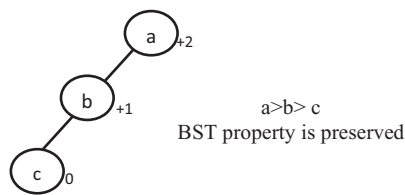


Figure 7.27c Insertion of element in an AVL tree

The balance factor of node a is +2 and hence does not satisfy the AVL property. We fix it with the following steps (RR rotation) (Figure 7.27d).

*RR rotation*

- Step 1: b becomes the new root.  
Step 2: a becomes the right child of b.  
Step 3: c continues to be the left child of b.

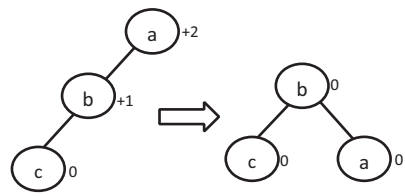


Figure 7.27d Insertion of element in an AVL tree

Both the AVL property and binary search tree property are restored.

Let us see what happens when the new node is the left child of the right sub-tree (see Figure 7.27e).

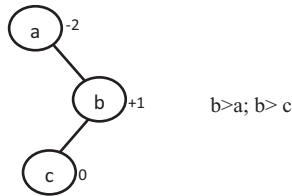


Figure 7.27e Insertion of element in an AVL tree

The balancing factor of  $a$  is 2 and hence AVL structure is not maintained. The new node is added as the left child of the right sub-tree. Now we have to be a little careful.

We initially make a rotation about  $b$  which yields, and then we make the familiar RR rotation to arrive at Figure 7.27f which satisfies both AVL and binary search tree properties

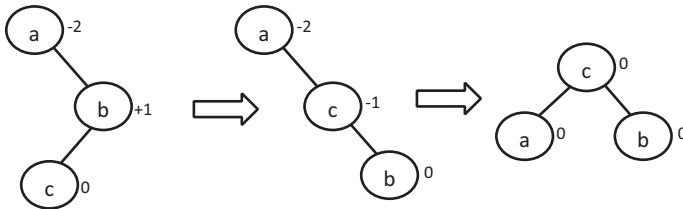


Figure 7.27f Insertion of element in an AVL tree

These two rotations together is called an *RL rotation*.

Similarly, we use *LR rotation* to convert the following binary search tree, where a new node is added to the right of a left sub-tree (Figure 7.27g). The sub-tree rooted at  $a$  does not satisfy the AVL property. To bring it back to

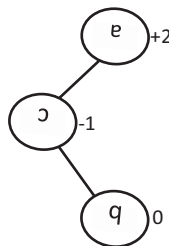


Figure 7.27g Insertion of element in an AVL tree

AVL structure, we first rotate about c, which makes b take the position of c, and c becomes the left child of b, and then rotate about a. This double rotation is called LR rotation.

We arrive at Figure 7.27h.

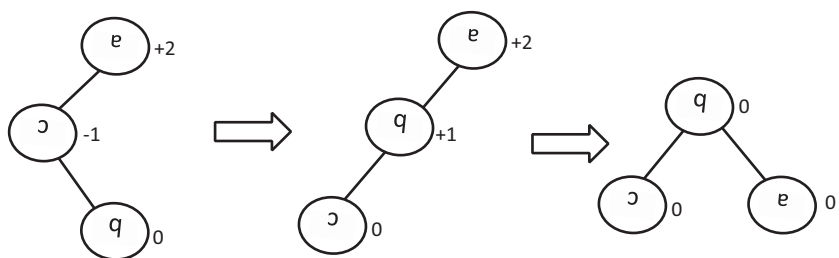


Figure 7.27h Insertion of element in an AVL tree

If we observe closely, we need two rotations if the balance factor at the root (where the balance factor is +2 or -2) and the balance factor of its child are of opposite signs. If the balance factor of the root is +2 and its child has balance factor -1, we first make a left rotation (which makes the sign of balance factors the same) and then a right rotation (LR rotation). If it is the other way (root -2 and its child +1), we first make a right rotation (making the signs of the balance factors the same) and then a left rotation (RL rotation).

Consider a full-sized example to construct a binary search tree with AVL property when the following elements are used to construct the tree (Figures 7.28a to 7.28f), read in order:

50, 17, 12, 23, 9, 14, 19, 72, 54, 76, 67

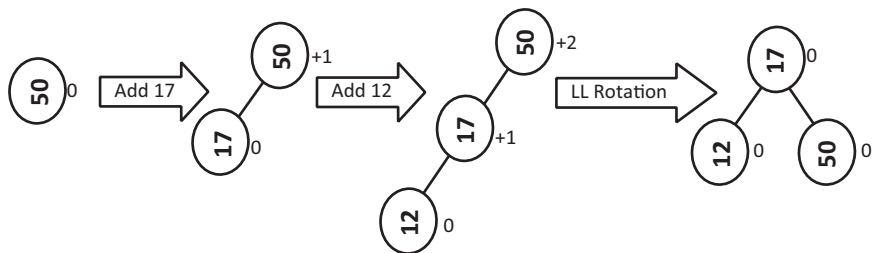


Figure 7.28a Insertion of element in an AVL tree

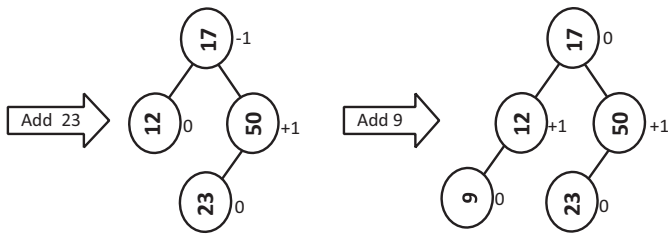


Figure 7.28b Insertion of element in an AVL tree

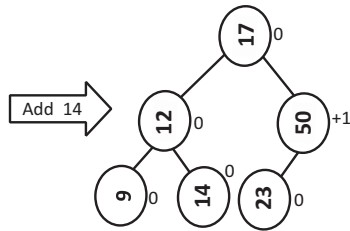


Figure 7.28c Insertion of element in an AVL tree

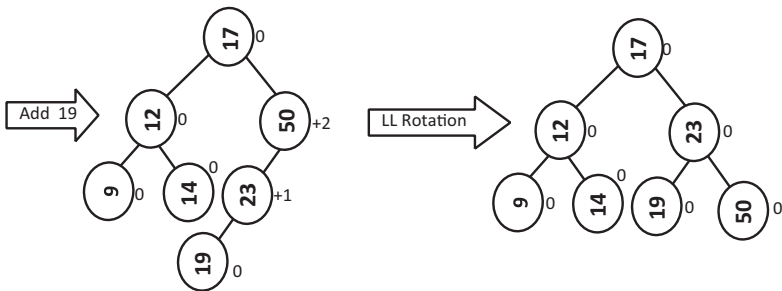


Figure 7.28d Insertion of element in an AVL tree

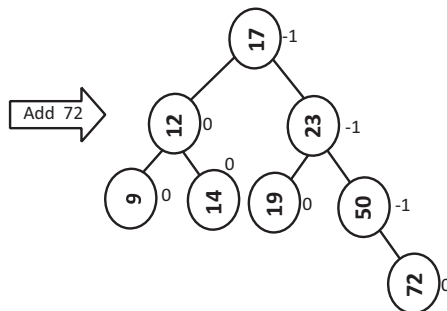


Figure 7.28e Insertion of element in an AVL tree

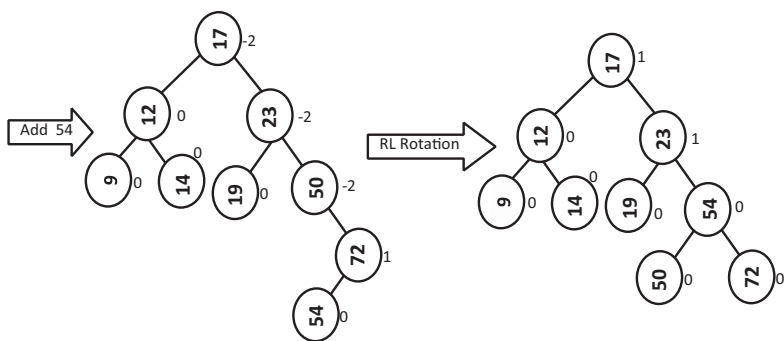


Figure 7.28f Insertion of element in an AVL tree

### 7.7.3 Deletion

As with insertion, the deletion of a node may cause the violation of AVL property. The sub-tree shown in Figure 7.29a is strictly in the AVL form.

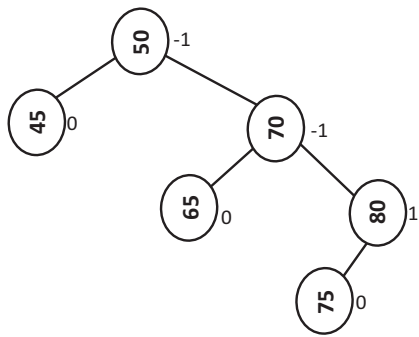


Figure 7.29a Deletion of element from an AVL tree

If we delete node 65, it is no longer an AVL tree since node 70 has balance factor (-2) and also node 50 has balance factor -2 as shown in Figure 7.29b.

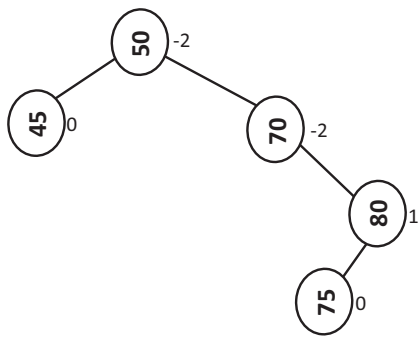


Figure 7.29b Deletion of element from an AVL tree

A restructuring is needed to make it an AVL tree. Unlike in the case of insertion where the restructuring is done locally, in deletion, the imbalance may percolate upward up to the root. Therefore, we follow the path backward up to the root, restructuring the sub-trees wherever needed using the rotations described earlier.

Let us take a concrete example. Consider deleting 8 from the AVL tree in Figure 7.29c.

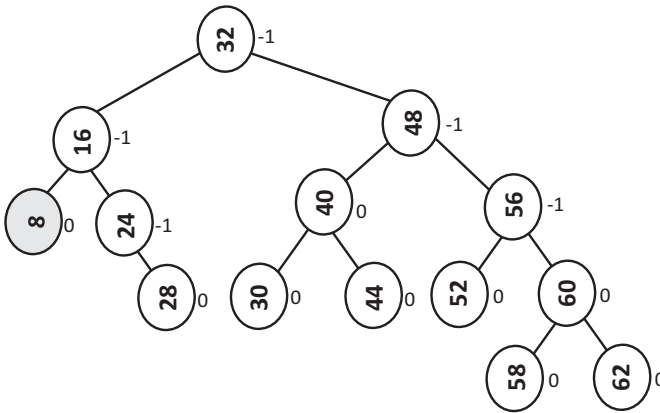


Figure 7.29c Deletion of element from an AVL tree

We start at 32, follow the path  $32 \rightarrow 16 \rightarrow 8$ . After deletion of 8, we move back to the root along the same path. Observe that the parent of 8, i.e., 16 has a balance factor  $-2$ . We locally restructure the sub-tree to obtain the tree in Figure 7.29d.

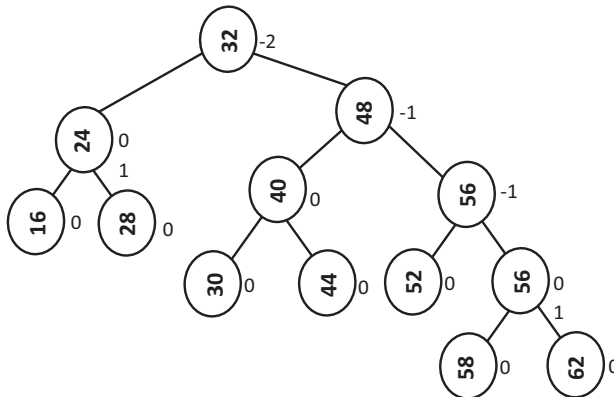


Figure 7.29d Deletion of element from an AVL tree



---

```

    # Update the balance factor and balance the tree
    balanceFactor = self.getBalance(root)
    if balanceFactor > 1:
        if key < root.left.key:
            return self.rightRotate(root)
        else:
            root.left = self.leftRotate(root.left)
            return self.rightRotate(root)

    if balanceFactor < -1:
        if key > root.right.key:
            return self.leftRotate(root)
        else:
            root.right = self.rightRotate(root.right)
            return self.leftRotate(root)

    return root

# Function to delete a node
def delete_node(self, root, key):

    # Find the node to be deleted and remove it
    if not root:
        return root
    elif key < root.key:
        root.left = self.delete_node(root.left, key)
    elif key > root.key:
        root.right = self.delete_node(root.right, key)
    else:
        if root.left is None:
            temp = root.right
            root = None
            return temp
        elif root.right is None:
            temp = root.left
            root = None
            return temp
        temp = self.getMinValueNode(root.right)
        root.key = temp.key
        root.right = self.delete_node(root.right,
                                     temp.key)

    if root is None:
        return root

    # Update the balance factor of nodes
    root.height = 1 + max(self.getHeight(root.left),
                          self.getHeight(root.right))

    balanceFactor = self.getBalance(root)

```

```
# Balance the tree
if balanceFactor > 1:
    if self.getBalance(root.left) >= 0:
        return self.rightRotate(root)
    else:
        root.left = self.leftRotate(root.left)
        return self.rightRotate(root)
if balanceFactor < -1:
    if self.getBalance(root.right) <= 0:
        return self.leftRotate(root)
    else:
        root.right = self.rightRotate(root.right)
        return self.leftRotate(root)
return root

# Function to perform left rotation
def leftRotate(self, z):
    y = z.right
    T2 = y.left
    y.left = z
    z.right = T2
    z.height = 1 + max(self.getHeight(z.left), self.
        getHeight(z.right))
    y.height = 1 + max(self.getHeight(y.left), self.
        getHeight(y.right))
    return y

# Function to perform right rotation
def rightRotate(self, z):
    y = z.left
    T3 = y.right
    y.right = z
    z.left = T3
    z.height = 1 + max(self.getHeight(z.left), self.
        getHeight(z.right))
    y.height = 1 + max(self.getHeight(y.left), self.
        getHeight(y.right))
    return y

# Get the height of the node
def getHeight(self, root):
    if not root:
        return 0
    return root.height

# Get balance factore of the node
def getBalance(self, root):
    if not root:
```

---

```

        return 0
    return self.getHeight(root.left) - self.getHeight
        (root.right)

def getMinValueNode(self, root):
    if root is None or root.left is None:
        return root
    return self.getMinValueNode(root.left)

# Print the tree
def display(self, currPtr, indent, last):
    if currPtr != None:
        print(indent,end="")
        if last:
            print("R----",end="")
            #sys.stdout.write("R----")
            indent += "        "
        else:
            print("L----",end="")
            #sys.stdout.write("L----")
            indent += "|        "
        print(currPtr.key)
        self.display(currPtr.left, indent, False)
        self.display(currPtr.right, indent, True)

myTree = AVLTree()
root = None
nums=[]
print("Enter 0 to exit Inserting Data into the AVL Tree ")
print("Enter the node of AVL Tree ")
x = int(input("Enter an element: "))
while(x!=0):
    nums.append(x)
    x = int(input("Enter an element: "))
nums=set(nums)
for num in nums:
    root = myTree.insert_node(root, num)
myTree.display(root, "", True)

key = int(input("Enter an element to Delete: "))
if(key not in nums):
    print(key,"Doesnot Exist")
else:
    root = myTree.delete_node(root, key)
    print("After Deletion: ")
    myTree.display(root, "", True)

```

## Output

```
Enter 0 to exit Inserting Data into the AVL Tree
Enter the node of AVL Tree
Enter an element: 5
Enter an element: 6
Enter an element: 2
Enter an element: 9
Enter an element: 1
Enter an element: 0
R----2
    L----1
    R----6
        L----5
        R----9
Enter an element to Delete: 5
After Deletion:
R----2
    L----1
    R----6
        R----9
```

## 7.8 SPLAY TREES

The main concern in balancing trees is to keep them from becoming lopsided and ideally allowing leaves to occur only at one or two levels. Therefore, if a newly arriving element disturbs the balance of the tree, a restructuring of the tree is done locally (AVL method). This restructuring is needed to reduce the average access time. However, not all elements are accessed with the same frequency. If an element at the tenth level of the tree is accessed infrequently, it may not be a worthwhile exercise to rebalance the tree. However, if the same element is frequently accessed, it makes a great difference if it is at the second level. The average access time will be greatly reduced if the frequently accessed elements are closer to the root. Therefore, the strategy in self-balancing trees should be to restructure the tree by moving up the elements frequently accessed.

One strategy to find the frequently accessed elements is to assume that the element being accessed has a good chance of being accessed again soon (a good example is the loop variable in a for loop). Therefore, it may be moved up the tree with no further restructuring.

Splaying is a method to move the recently accessed element to the root. This move-to-the-root-strategy has three cases depending on the links between the child, parent and the grandparent. Let X be the recently accessed element and let P be its parent and G its grandparent.

Case 1: X's parent P is the root. Perform a single rotation about the root (P) making X the root. This is called a “zig” step (also called singular splay). It is the same as a single right rotation (Figure 7.30).

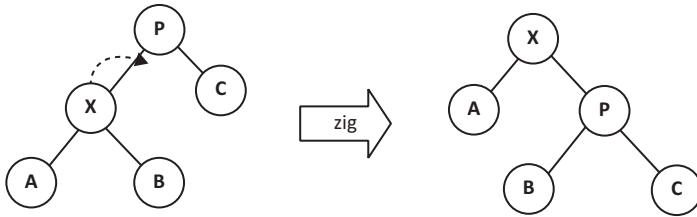


Figure 7.30 Zig operation in splay tree

Its mirror situation is known as a “zag” step. It is the same as a single left rotation (Figure 7.31).

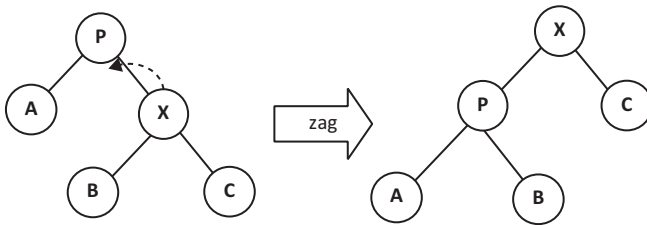


Figure 7.31 Zag operation in splay tree

Case 2: X is the left child of its parent P and P is the left child of its parent G (Figure 7.32). Or the mirror situation where X is the right child of its parent P and P is the right child of its parent G. This is called a homogeneous configuration. First rotate P about G and then rotate X about P. This is called

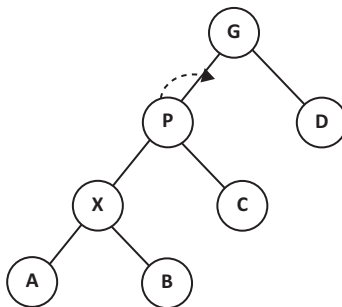


Figure 7.32 Zig-zig operation in splay tree

the “zig-zig” step. It is also known as a double zig (double right rotation) (Figure 7.33a).

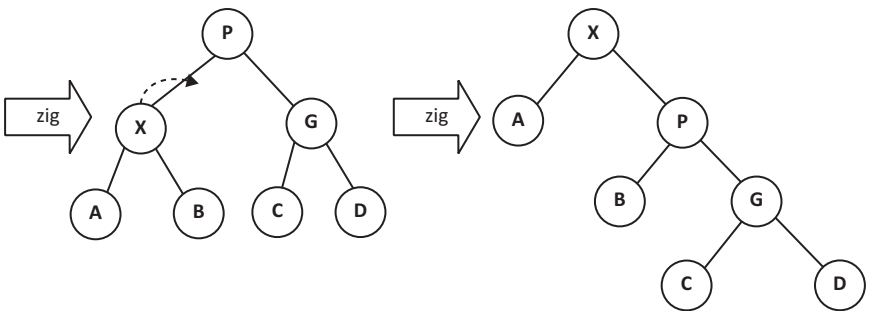


Figure 7.33a Zig-zig operation in splay tree

Same as Figure 7.33b.

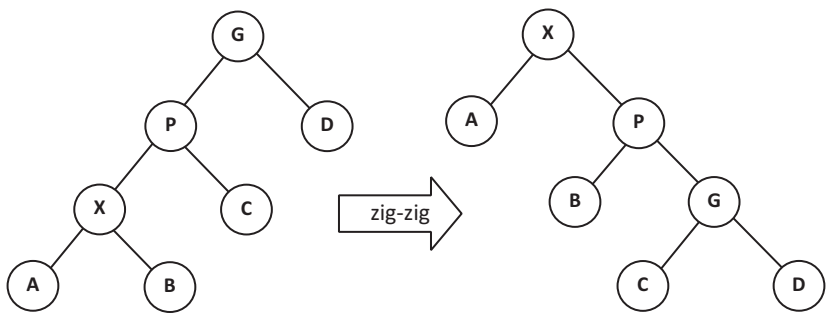


Figure 7.33b Zig-zig operation in splay tree

Its mirror situation is known as a “zag-zag” step. It is the same as a double left rotation (Figure 7.34a).

Same as Figure 7.34b.

Case 3: X is the left child of its parent P and P is the right child of its parent G. Or its mirror situation where X is the right child of P and P is the left child of G. This is called a heterogeneous configuration. First, perform a rotation of X about P and then about G. This is called a “zig-zag” step (Figure 7.35a).

Same as Figure 7.35b.

Its mirror situation is known as a “zag-zig” step. It is the same as left-right rotation (Figure 7.36a).

Same as Figure 7.36b.

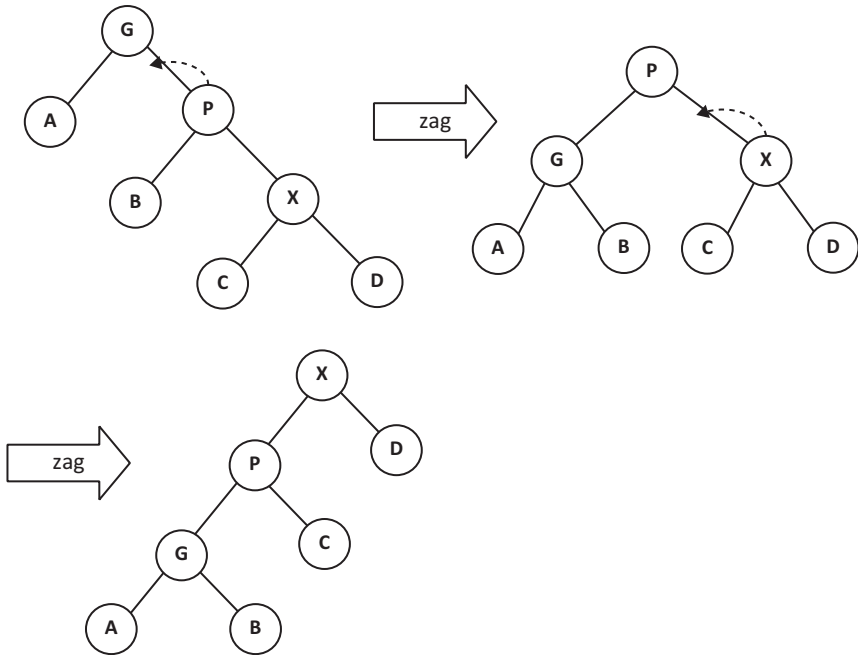


Figure 7.34a Zig-zig operation in splay tree

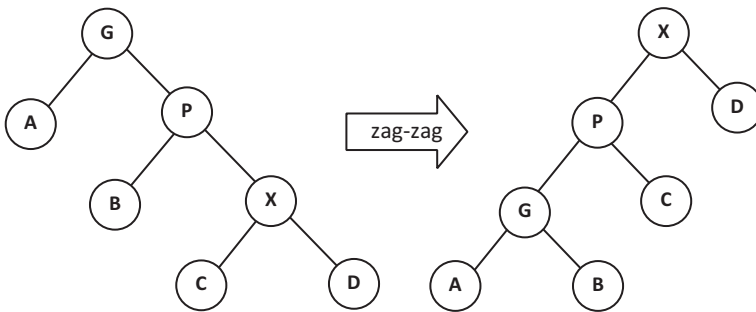


Figure 7.34b Zig-zag operation in splay tree

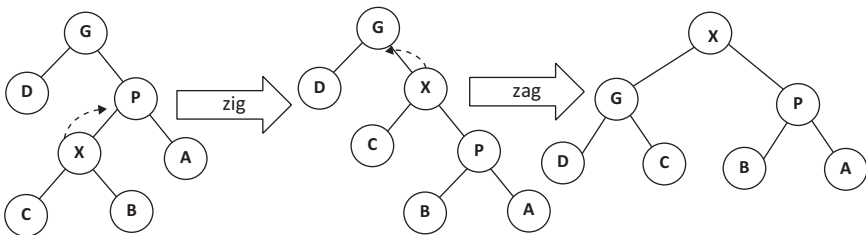


Figure 7.35a Zig-zag operation in splay tree

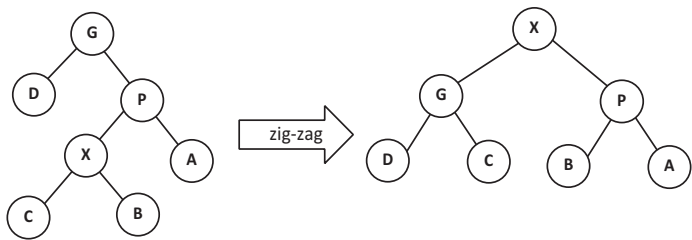


Figure 7.35b Zig-zag operation in splay tree

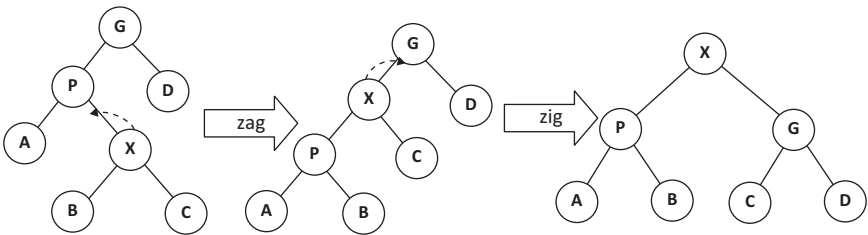


Figure 7.36a Zag-zig operation in splay tree

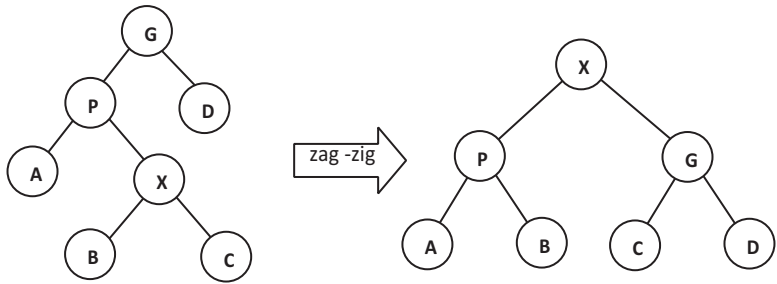


Figure 7.36b Zag-zig operation in splay tree

### 7.8.1 Insertion: Interchange B and C

1. Insert the element X as we do in a binary search tree.
2. Perform a splay (one of the three cases may occur).
3. The newly inserted element becomes the root.

**Deletion:** We use the same method as in a normal binary search tree. We splay the parent of the deleted node.

**Example:** With 5, 10, 15, 6, 9, 7, 11, 4 form a splay tree inserting the elements in the same order (Figures 7.37a to 7.37g).

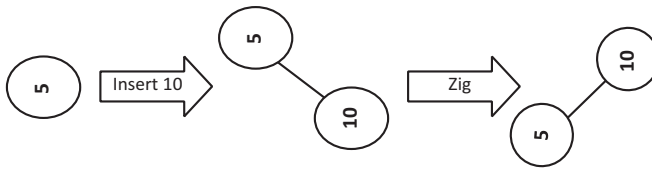


Figure 7.37a Insertion of 10 into a splay tree

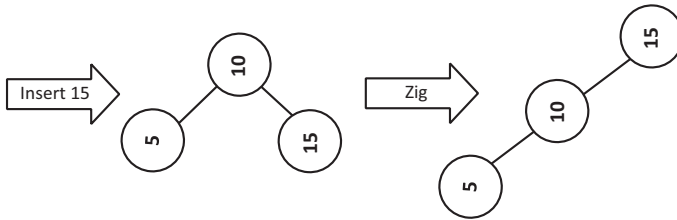


Figure 7.37b Insertion of 15 into a splay tree

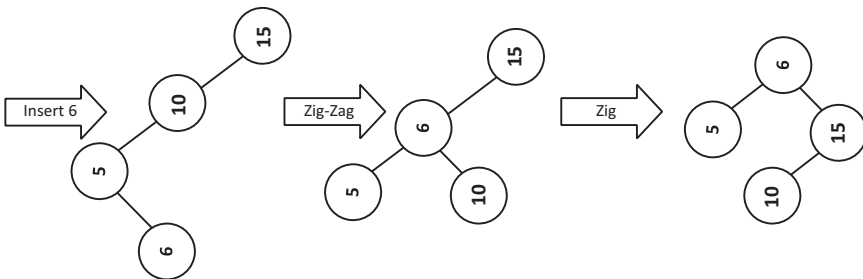


Figure 7.37c Insertion of 6 into a splay tree

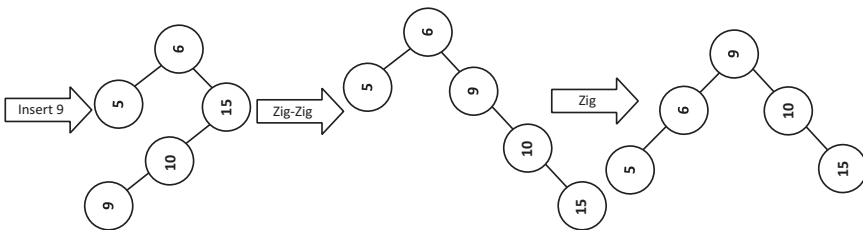


Figure 7.37d Insertion of 9 into a splay tree

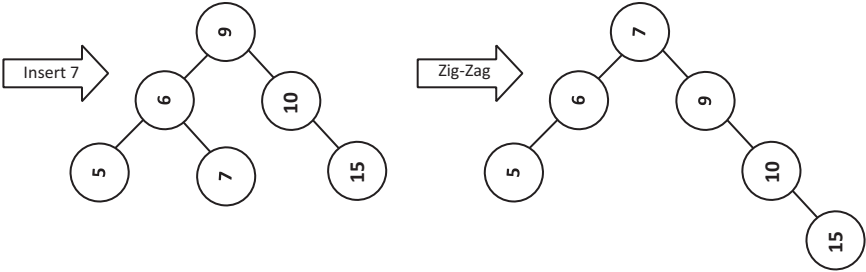


Figure 7.37e Insertion of 7 into a splay tree

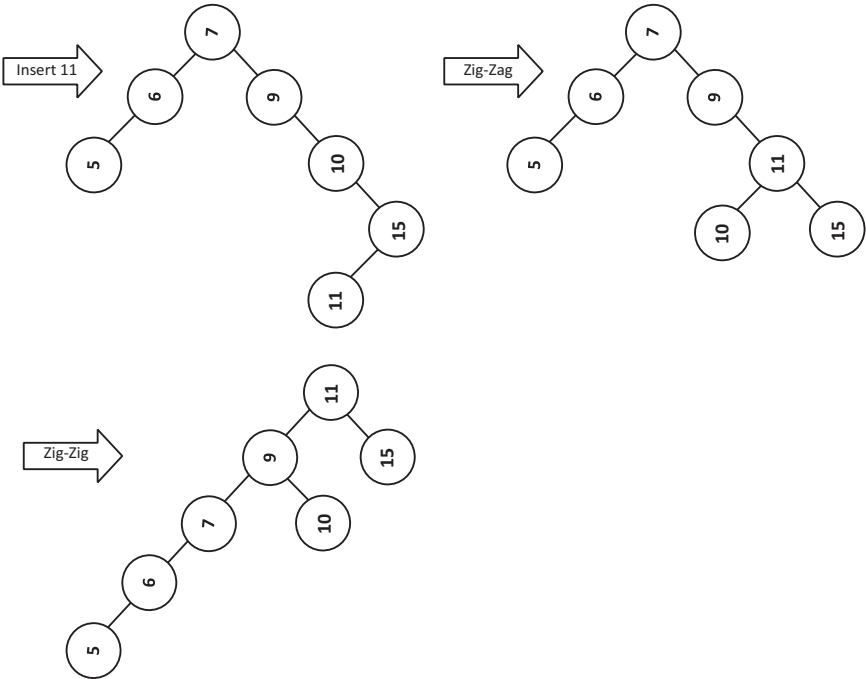


Figure 7.37f Insertion of 11 into a splay tree

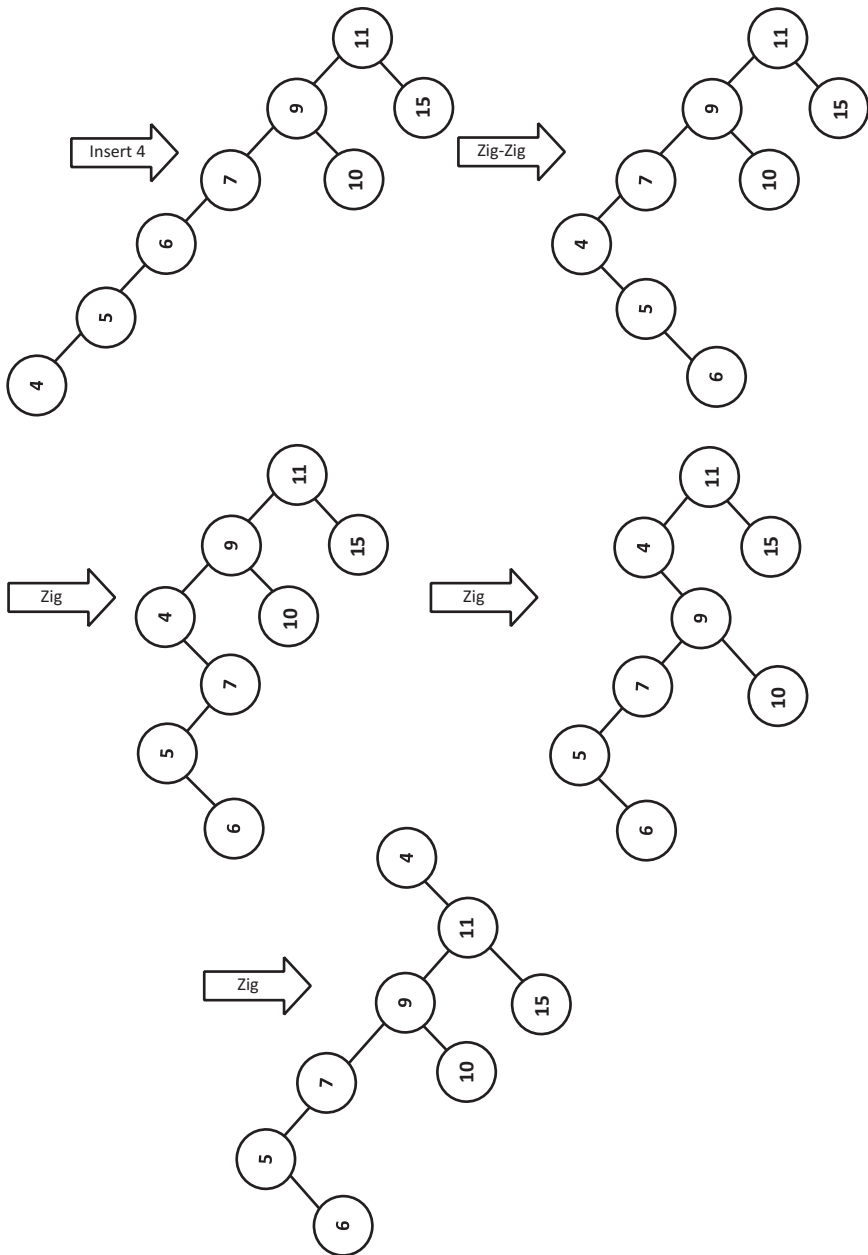


Figure 7.37g Insertion of 4 into a splay tree



- 
4. If a new node is inserted as the left child to a node that is the right child of its parent, then we make an
- a. RL rotation
  - b. LR rotation
  - c. LL rotation
  - d. RR rotation
5. We need a double rotation
- a. When the parent of the newly inserted and its grandparent differ in sign in their balance factors
  - b. When the balance factors are same
  - c. When both of them are in the same sub-tree
  - d. None of the above
6. In a splay tree, the newly accessed node is
- a. A leaf node
  - b. A node with only right child
  - c. A node with only left child
  - d. The root
7. If the key values 2, 3, 7, 8, 9, 1 are inserted into a splay tree, the root contains
- a. 2
  - b. 7
  - c. 1
  - d. 9
8. Why is a splay tree called the perfect tree?
- a. Space efficiency
  - b. Searching is quick
  - c. Easier to program and fast access time
  - d. Easy to construct
9. Why are splay trees preferred?
- a. Easier to program and faster access to recently accessed items
  - b. Space efficiency
  - c. Difficult to program
  - d. Quick searching
10. Which of the following rotation in AVL tree known as single rotation?
- a. LL and RR
  - b. LL and LR
  - c. LR and RL
  - d. RR and RL

11. A binary search tree is generated by inserting integers one by one in a particular order. Which traversal prints the nodes in the same order?
- a. Pre-order
  - b. In-order
  - c. Post-order
  - d. Level-order
12. What is the height of a complete binary tree with  $n$  nodes?
- a.  $\log(n+1) - 1$
  - b.  $\log n$
  - c.  $\log(n+1) + 1$
  - d.  $\log n - 1$
13. What is a splay operation?
- a. Moving leaf node
  - b. Moving root to leaf
  - c. Moving parent to child node
  - d. Moving node to root
14. A full binary tree with  $2n + 1$  nodes contains
- a.  $n$  leaf nodes
  - b.  $n$  internal nodes
  - c.  $n - 1$  leaf nodes
  - d.  $n - 1$  internal nodes
15. The minimum number of key values in a node of a B-tree with  $m+1$  links is
- a.  $\lceil m/2 \rceil$
  - b.  $\lfloor m/2 \rfloor$
  - c.  $m+1$
  - d.  $m-1$
16. The minimum number of key values in the root in a B-tree is
- a. 2
  - b. 3
  - c. 4
  - d. 5
17. The disadvantage of splay tree is
- a. Splay operations are difficult
  - b. Splay tree performs unnecessary splay when a node is only being read
  - c. No disadvantage
  - d. The height of a splay tree can be linear when accessing elements in non-decreasing order
18. The advantage of a B-tree is
- a. The number of disc accesses is reduced
  - b. The tree is balanced
  - c. Both of them
  - d. None of them

19. Which of the following is a self-adjusting or self-balancing binary search tree?

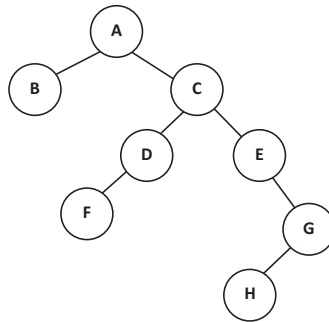
- a. Splay tree
- b. Red-black tree
- c. AVL tree
- d. All of the above

20. If 2000 numbers are used to construct a binary search tree, the maximum height of the tree is

- a. 1
- b. 11
- c. 1999
- d. 2000

### Descriptive questions

1. List the nodes in the binary tree given below in
  - i. Pre-order traversal
  - ii. In-order traversal
  - iii. Post-order traversal



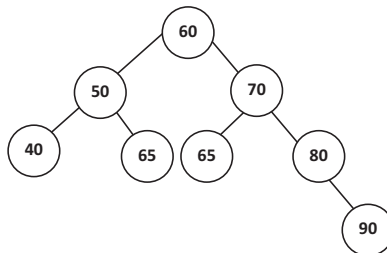
2. The nodes visited in the pre-order traversal and in-order traversal are given below.

Pre-order: ABCDEFGHIJKLM

In-order: CEDFBAHJIKGML

Draw the binary tree.

3. Insert node 100 in the following AVL tree. The resulting tree must be an AVL tree.



What is the balance factor of the root after insertion?

4. Starting from an empty AVL tree, insert the following key values:  
24, 39, 35, 47, 58, 36, 71, 100 in that order

**Answers to true–false questions**

1. True
2. False
3. True
4. True
5. False

**Answers to fill-in-the-blank questions**

1. height
2.  $2^{i-1}$
3. maximum level
4. leaf node
5. left-skewed tree

**Answers to multiple-choice questions**

- |     |   |     |   |     |   |     |   |     |   |
|-----|---|-----|---|-----|---|-----|---|-----|---|
| 1.  | c | 2.  | a | 3.  | b | 4.  | a | 5.  | a |
| 6.  | d | 7.  | d | 8.  | c | 9.  | a | 10. | a |
| 11. | d | 12. | b | 13. | d | 14. | b | 15. | a |
| 16. | a | 17. | d | 18. | d | 19. | d | 20. | c |